

从0到1手写实现一个分布式RPC框架

前言

为什么要自己写一个RPC框架，从个人成长上说，如果一个程序员能清楚的了解RPC框架所具备的要素，掌握RPC框架中涉及的服务注册发现、负载均衡、序列化协议、RPC通信协议、Socket通信、异步调用、熔断降级等技术，可以全方位的提升基本素质。虽然也有相关源码，但是只看源码容易眼高手低，动手写一个才是自己真正掌握这门技术的最优路径。

一 什么是RPC

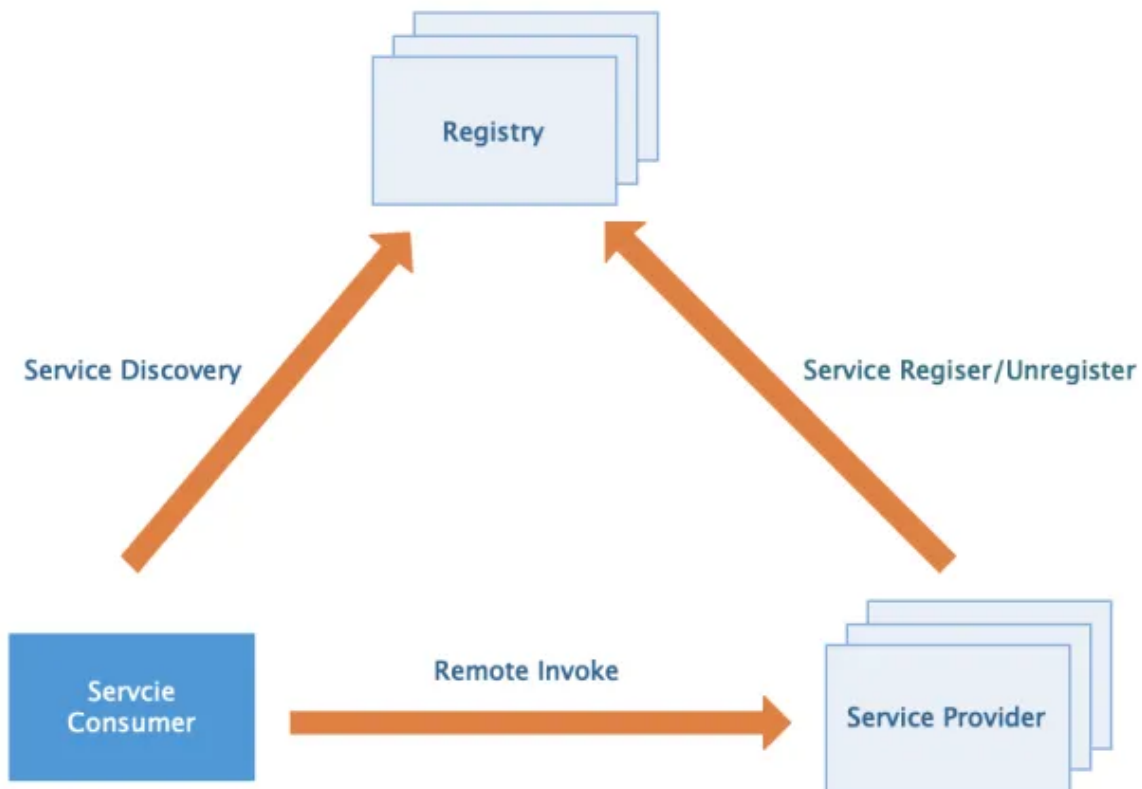
RPC (Remote Procedure Call) 远程过程调用，简言之就是像调用本地方法一样调用远程服务。目前外界使用较多的有gRPC、Dubbo、Spring Cloud等。相信大家对RPC的概念都已经很熟悉了，这里不做过多介绍。

二 分布式RPC框架要素

一款分布式RPC框架离不开三个基本要素：

- 服务提供方 Service Provider
- 服务消费方 Service Consumer
- 注册中心 Registry

围绕上面三个基本要素可以进一步扩展服务路由、负载均衡、服务熔断降级、序列化协议、通信协议等等。



1 注册中心

主要是用来完成服务注册和发现的工作。虽然服务调用是服务消费方直接发向服务提供方的，但是现在服务都是集群部署，服务的提供者数量也是动态变化的，所以服务的地址也就无法预先确定。因此如何发现这些服务就需要一个统一注册中心来承载。

2 服务提供方（RPC服务端）

其需要对外提供服务接口，它需要在应用启动时连接注册中心，将服务名及其服务元数据发往注册中心。同时需要提供服务服务下线的机制。需要维护服务名和真正服务地址映射。服务端还需要启动Socket服务监听客户端请求。

3 服务消费方（RPC客户端）

客户端需要有从注册中心获取服务的基本能力，它需要在应用启动时，扫描依赖的RPC服务，并为其生成代理调用对象，同时从注册中心拉取服务元数据存入本地缓存，然后发起监听各服务的变动做到及时更新缓存。在发起服务调用时，通过代理调用对象，从本地缓存中获取服务地址列表，然后选择一种负载均衡策略筛选出一个目标地址发起调用。调用时会请求数据进行序列化，并采用一种约定的通信协议进行socket通信。

三 技术选型

1 注册中心

目前成熟的注册中心有Zookeeper，Nacos，Consul，Eureka，它们的主要比较如下：

	Zookeeper	Nacos	Consul	Eureka
一致性协议	CP	CP+AP	CP	AP
雪崩保护	无	有	无	有
多数据中心	不支持	支持	支持	支持
自动注销实例	支持	支持	支持	支持

本实现中支持了两种注册中心Nacos和Zookeeper，可根据配置进行切换。

2 IO通信框架

本实现采用Netty作为底层通信框架，Netty是一个高性能事件驱动型的非阻塞的IO(NIO)框架。

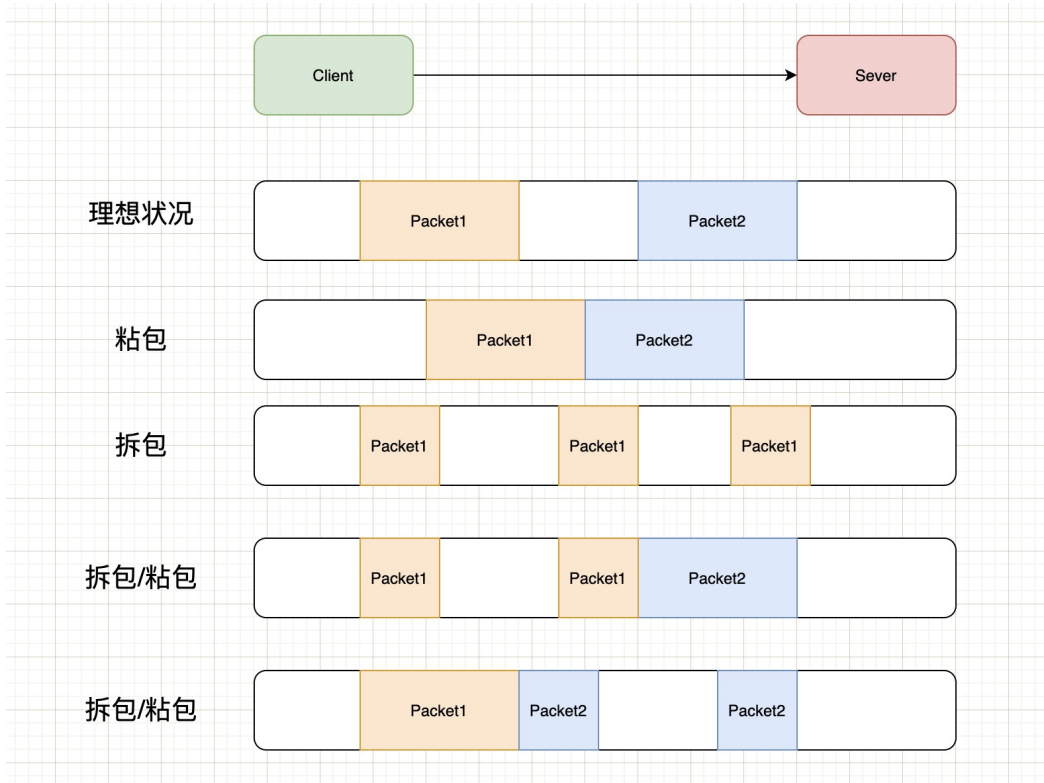
3 通信协议

TCP通信过程中会根据TCP缓冲区的实际情况进行包的划分，所以在业务上认为一个完整的包可能会被TCP拆分成多个包进行发送，也有可能把多个小的包封装成一个大的数据包发送，这就是所谓的TCP粘包和拆包问题。所以需要对发送的数据包封装到一种通信协议里。

因为TCP是面向流，没有边界，而操作系统在发送TCP数据时，会通过缓冲区来进行优化，例如缓冲区为1024个字节大小。

如果一次请求发送的数据量比较小，没达到缓冲区大小，TCP则会将多个请求合并为同一个请求进行发送，这就形成了粘包问题。

如果一次请求发送的数据量比较大，超过了缓冲区大小，TCP就会将其拆分为多次发送，这就是拆包。



正常的理想情况，两个包恰好满足TCP缓冲区的大小或达到TCP等待时长，分别发送两个包；
粘包：两个包较小，间隔时间短，发生粘包，合并成一个包发送；
拆包：一个包过大，超过缓存区大小，拆分成两个或多个包发送；
拆包和粘包：Packet1过大，进行了拆包处理，而拆出去的一部分又与Packet2进行粘包处理。

业界的主流协议的解决方案可以归纳如下：

- 1. 消息定长，例如每个报文的大小为固定长度100字节，如果不够用空格补足。
- 2. 在包尾特殊结束符进行分割。
- 3. 将消息分为消息头和消息体，消息头中包含表示消息总长度（或者消息体长度）的字段。

很明显1，2都有些局限性，本实现采用方案3，具体协议设计如下：



- 第一个字节是魔法数，比如我定义为0X35。

- 第二个字节代表协议版本号，以便对协议进行扩展，使用不同的协议解析器。
- 第三个字节是请求类型，如0代表请求1代表响应。
- 第四个字节表示消息长度，即此四个字节后面此长度的内容是消息content。

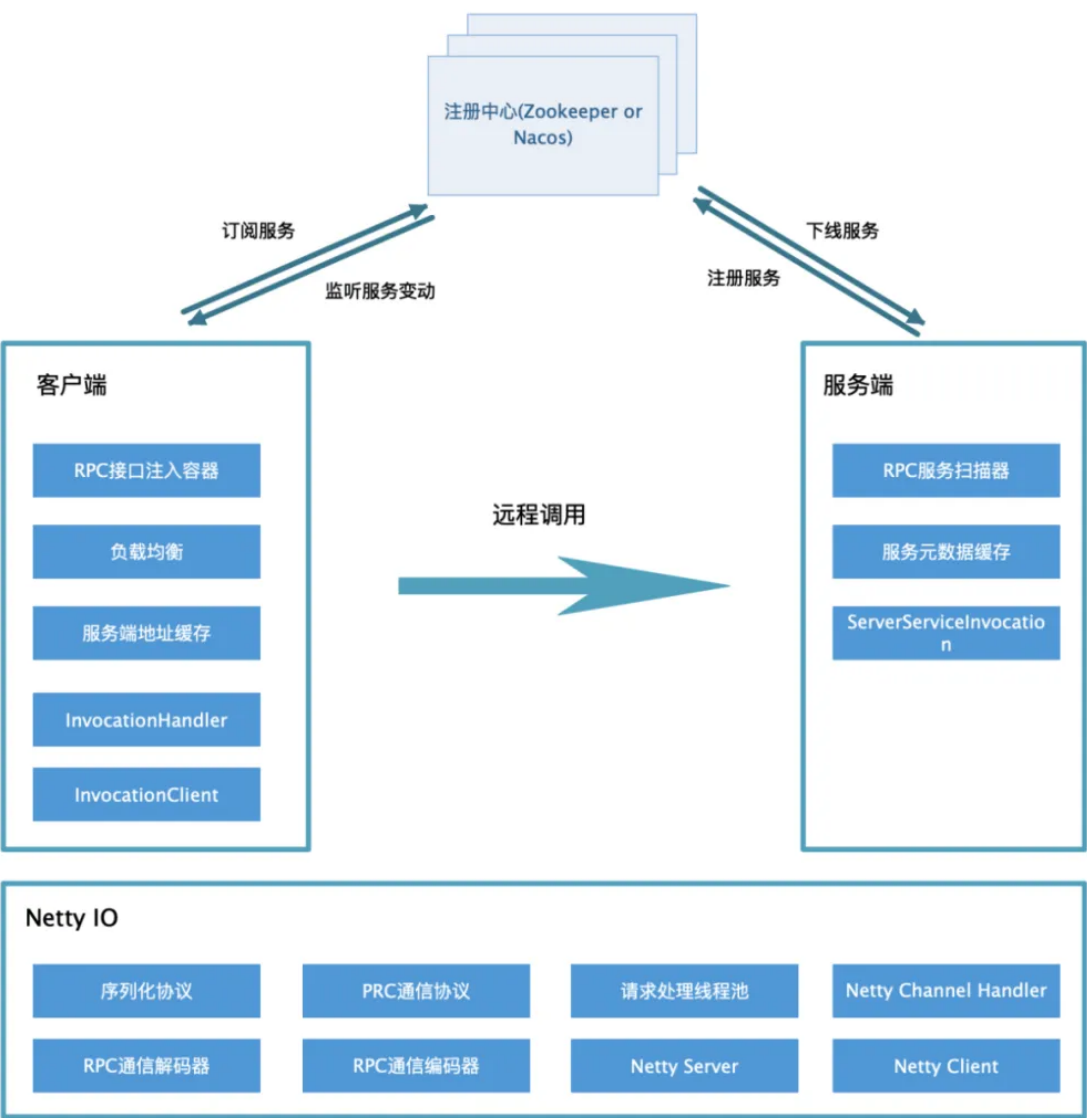
4 序列化协议

本实现支持3种序列化协议，JavaSerializer、Protobuf及Hessian可以根据配置灵活选择。建议选用Protobuf，其序列化后码流小性能高，非常适合RPC调用，Google自家的gRPC也是用其作为通信协议。

5 负载均衡

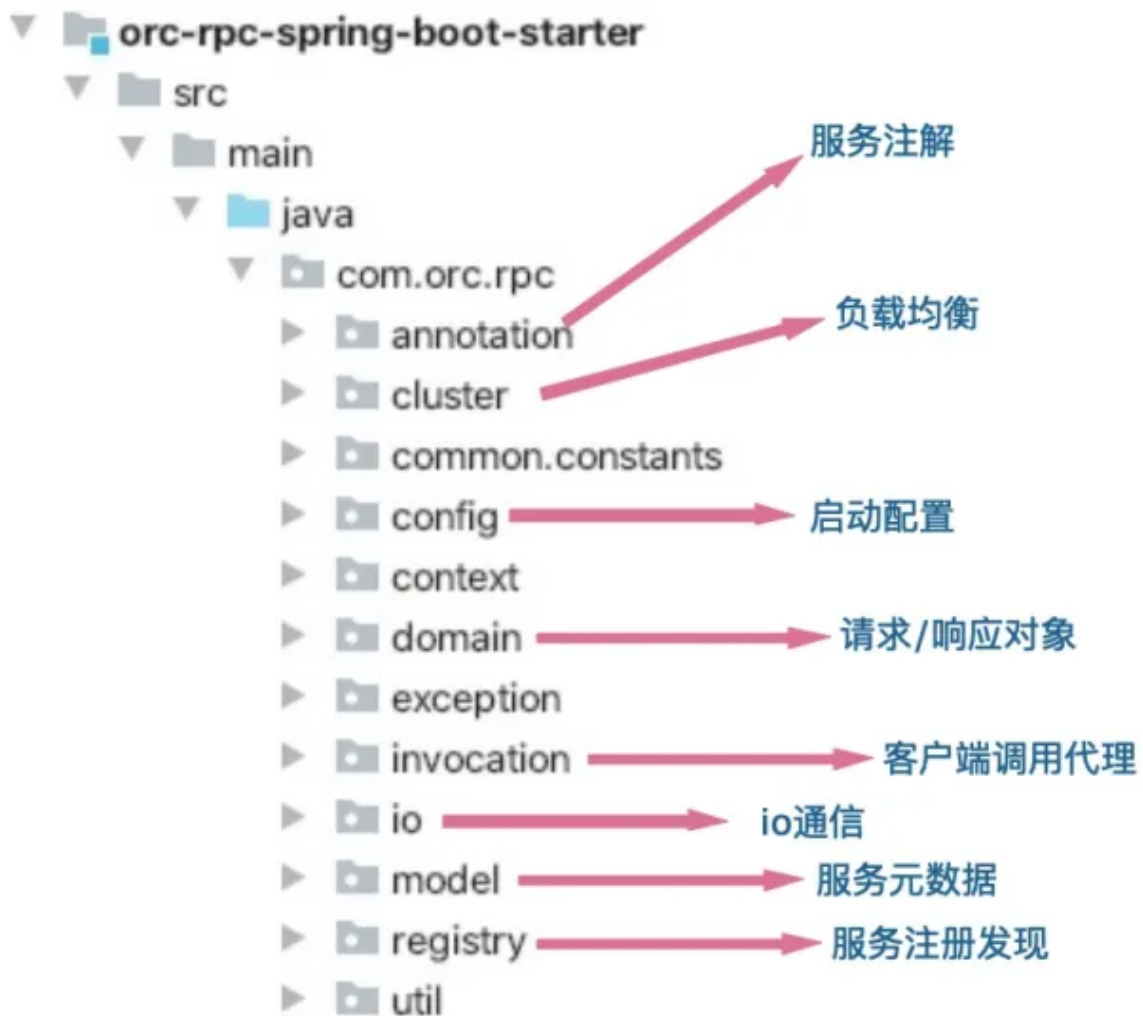
本实现支持两种主要负载均衡策略，随机和轮询，其中他们都支持带权重的随机和轮询，其实也就是四种策略。

四 整体架构



五 实现

项目总体结构:

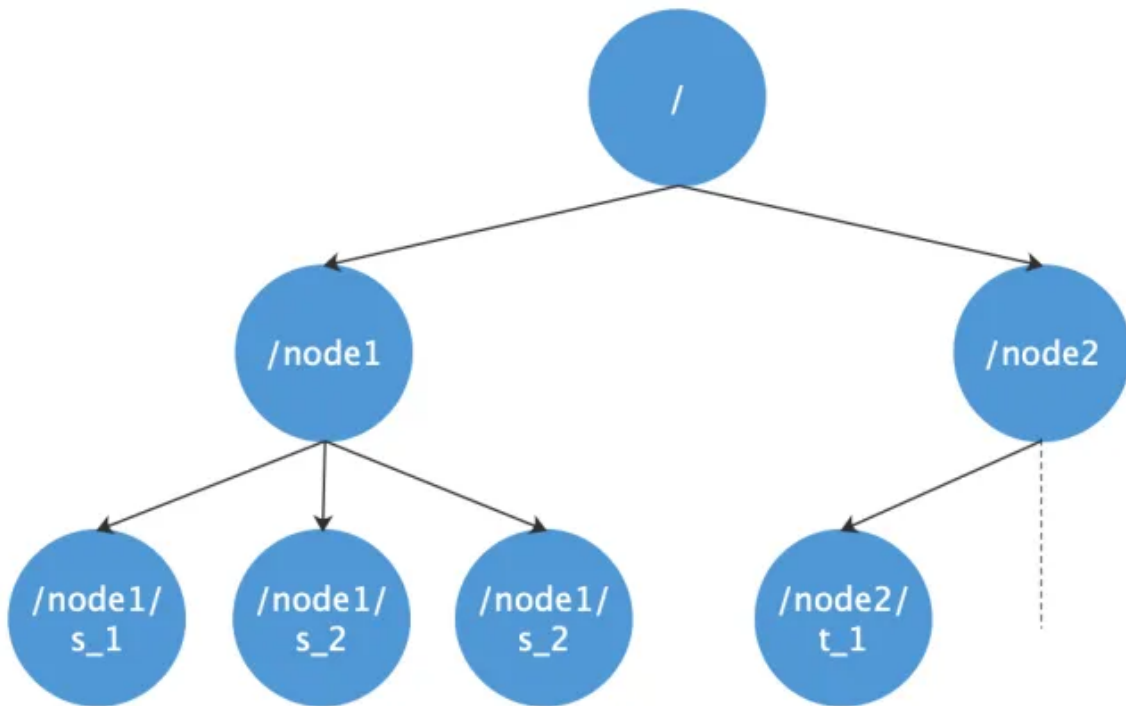


1 服务注册发现



Zookeeper

Zookeeper采用节点树的数据模型，类似linux文件系统，`/`，`/node1`，`/node2` 比较简单。



Zookeeper节点类型是Zookeeper实现很多功能的核心原理，分为持久节点临时节点、顺序节点三种类型的节点。

我们采用的是对每个服务名创建一个持久节点，服务注册时实际上就是在zookeeper中该持久节点下创建了一个临时节点，该临时节点存储了服务的IP、端口、序列化方式等。

```

/**
 * 在zk上创建服务节点
 *
 * @param serviceURL
 * @return void
 */
private void createZookeeperServiceNode(ServiceURL serviceURL) {
    String urlJson = JSON.toJSONString(serviceURL);
    try {
        urlJson = URLEncoder.encode(urlJson, UTF_8);
    } catch (UnsupportedEncodingException e) {
        log.error("UnsupportedEncodingException utf8 encode", e);
    }

    String servicePath = ServiceUtils.getRegisterServiceParentPath(serviceURL.getServiceId());
    // 创建服务节点
    if (!zkClient.exists(servicePath)) {
        zkClient.createPersistent(servicePath, createParents: true); // 服务名路径持久节点
    }
    String urlPath = servicePath + SERVICE_PATH_DELIMITER + urlJson;
    if (zkClient.exists(urlPath)) {
        // 删除之前的节点
        zkClient.delete(urlPath);
    }
    // 创建一个临时节点，会话失效即被清理
    zkClient.createEphemeral(urlPath); // 持久节点下创建临时节点
}

```

客户端获取服务时通过获取持久节点下的临时节点列表，解析服务地址数据：

```

@Override
public List<ServiceURL> getServiceList(ServiceMetadata serviceMetadata) {
    String serviceId = ServiceUtils.getServiceId(serviceMetadata);
    List<ServiceURL> services = ClientServiceDiscoveryCache.get(serviceId);

    if (CollectionUtils.isEmpty(services)) {
        String lockKey = ClientServiceDiscoveryCache.getCacheLockKey(serviceId);
        synchronized (lockKey) {
            services = ClientServiceDiscoveryCache.get(serviceId);
            if (CollectionUtils.isEmpty(services)) {
                String servicePath = ServiceUtils.getRegisterServiceParentPath(serviceId);
                List<String> children = zkClient.getChildren(servicePath);
                if (CollectionUtils.isEmpty(children)) {
                    throw new OrcaRpcException("No rpc provider " + serviceId + " available!");
                }
                services = getAndSetServiceCache(serviceId, children);
            }
        }
    }

    return services;
}

```

获取服务名持久节点下的临时节点列表

客户端监听服务变化:

```

public void subscribeServiceChange(ServiceMetadata serviceMetadata) {
    String serviceId = ServiceUtils.getServiceId(serviceMetadata);
    zkClient.subscribeChildChanges(ServiceUtils.getRegisterServiceParentPath(serviceId),
        new IZkChildListener() {
            @Override
            public void handleChildChange(String parentPath, List<String> currentChilds) throws Exception {
                log.debug("parentPath:{}, currentChilds:{}", parentPath, currentChilds);
                // 只要子节点有改动就清空缓存
                String[] arr = parentPath.split(regex: "/");
                String serviceId = arr[2];

                ZookeeperRegistry.this.serviceChangeCallBack(serviceId, currentChilds);
            }
        });
}

```

通过服务名持久节点路径订阅变化

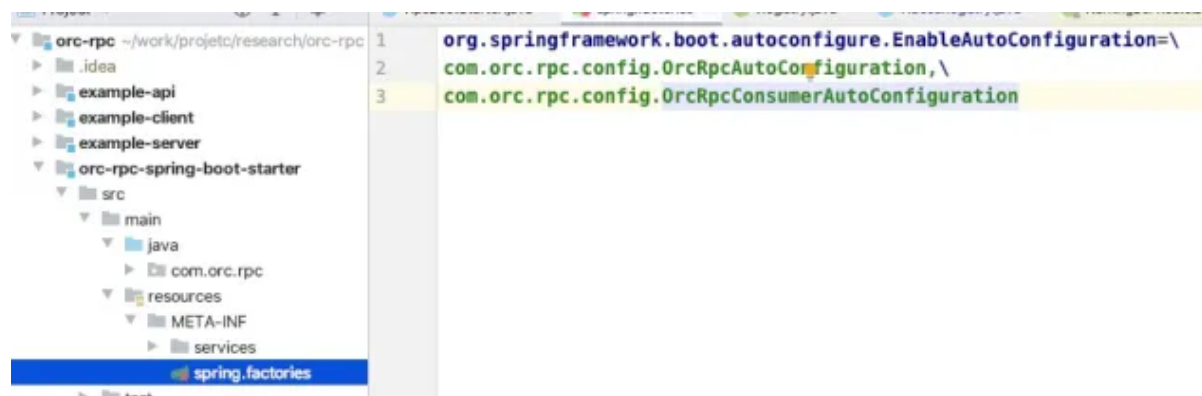
Nacos

Nacos是阿里开源的微服务管理中间件，用来完成服务之间的注册发现和配置中心，相当于Spring Cloud的Eureka+Config。

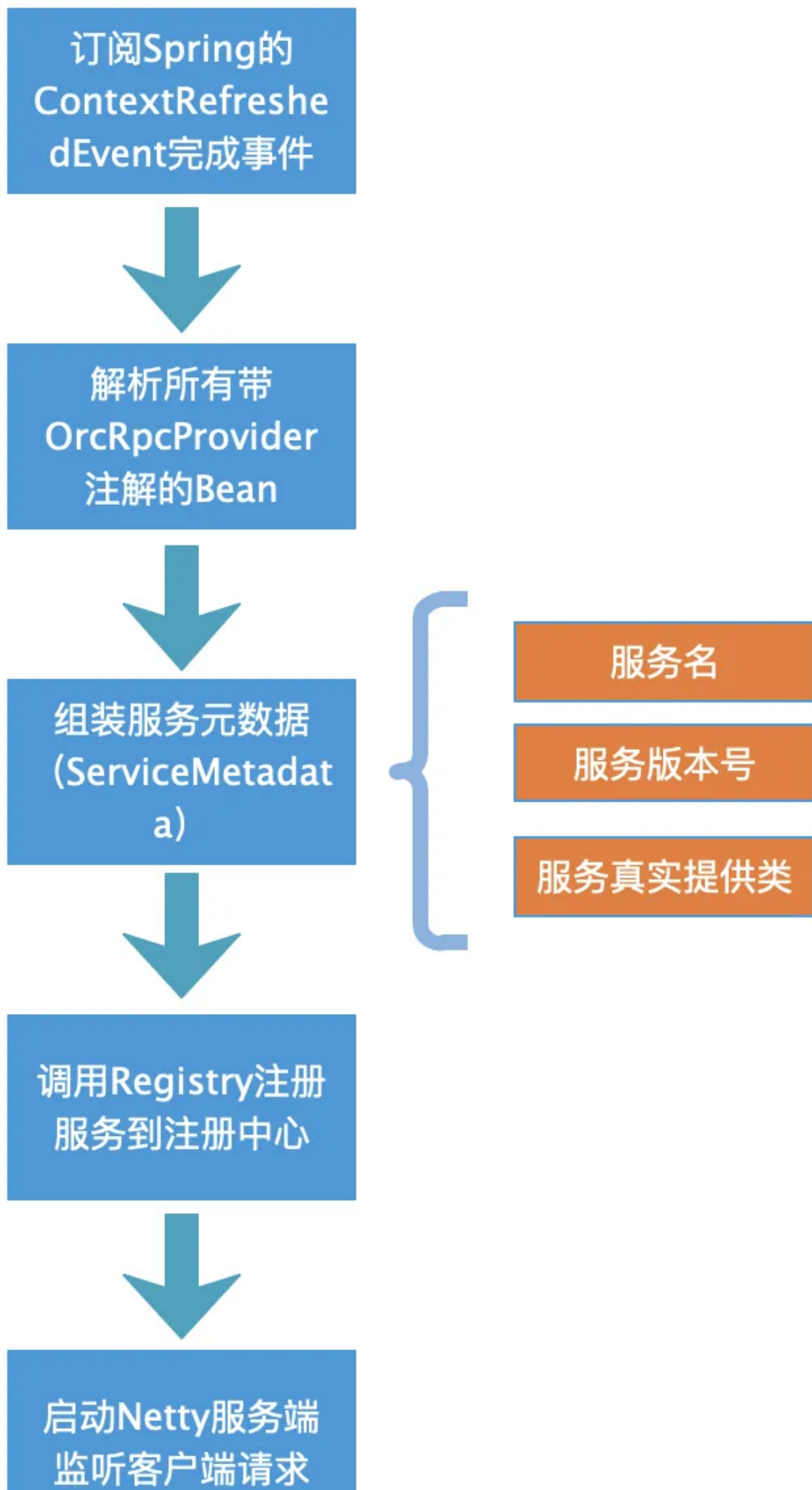
不像Zookeeper需要利用提供的创建节点特性来实现注册发现，Nacos专门提供了注册发现功能，所以其使用更加方便简单。主要关注NamingService接口提供的三个方法registerInstance、getAllInstances、subscribe；registerInstance用来完成服务端服务注册，getAllInstances用来完成客户端服务获取，subscribe用来完成客户端服务变动监听，这里就不多做介绍，具体可参照实现源码。

2 服务提供方 Service Provider

在自动配置类OrcaRpcAutoConfiguration完成注册中心和RPC启动类（RpcBootStarter）的初始化：



服务端的启动流程如下：



RPC启动 (RpcBootStarter) :

```
public class RpcBootStarter implements ApplicationListener<ContextRefreshedEvent> {

    private Registry serviceRegistry;

    private OrcRpcProperties orcRpcProperties;

    public RpcBootStarter(Registry serviceRegistry,
        OrcRpcProperties orcRpcProperties) {
        this.serviceRegistry = serviceRegistry; → Zookeeper或Nacos
        this.orcRpcProperties = orcRpcProperties;
    }

    /**
     * spring启动完成后执行
     *
     * @param event
     * @return void
     */
    @Override
    public void onApplicationEvent(ContextRefreshedEvent event) { → Spring容器初始化完成触发
        if (Objects.isNull(event.getApplicationContext().getParent())){
            ApplicationContext context = event.getApplicationContext();
            // 提供服务
            registerService(context);
            // 订阅服务
            subscribeService(context);
        }
    }

    private void registerService(ApplicationContext context) {
        Map<String, Object> beanMap = context.getBeansWithAnnotation(OrcRpcProvider.class);
        if (beanMap.size() > 0) {
            for (Object obj : beanMap.values()) {
                Class<?> clazz = obj.getClass();
                try {
                    OrcRpcProvider orcRpcProvider = clazz.getAnnotation(OrcRpcProvider.class);
                    if (StringUtils.isEmpty(orcRpcProvider.name())) {
                        throw new RuntimeException(
                            "orc rpc service class: " + clazz.getName() + " need service name config");
                    }
                    ServiceMetadata serviceMetadata = new ServiceMetadata();
                    serviceMetadata.setName(orcRpcProvider.name());
                    serviceMetadata.setVersion(orcRpcProvider.version());
                    serviceMetadata.setClazz(clazz);
                    serviceRegistry.register(serviceMetadata); → 调用注册中心注册服务
                } catch (Exception e) {
                    log.error("orc rpc service class: {}, register error", clazz.getName(), e);
                    throw new RuntimeException(
                        "orc rpc service class: " + clazz.getName() + " register error");
                }
            }
        }

        (Thread) run() → {
            OrcRpcServerContainer.initServer(orcRpcProperties).start(); → 启动netty服务端
        }.start(); → 监听客户客户端请求
    }
}
```

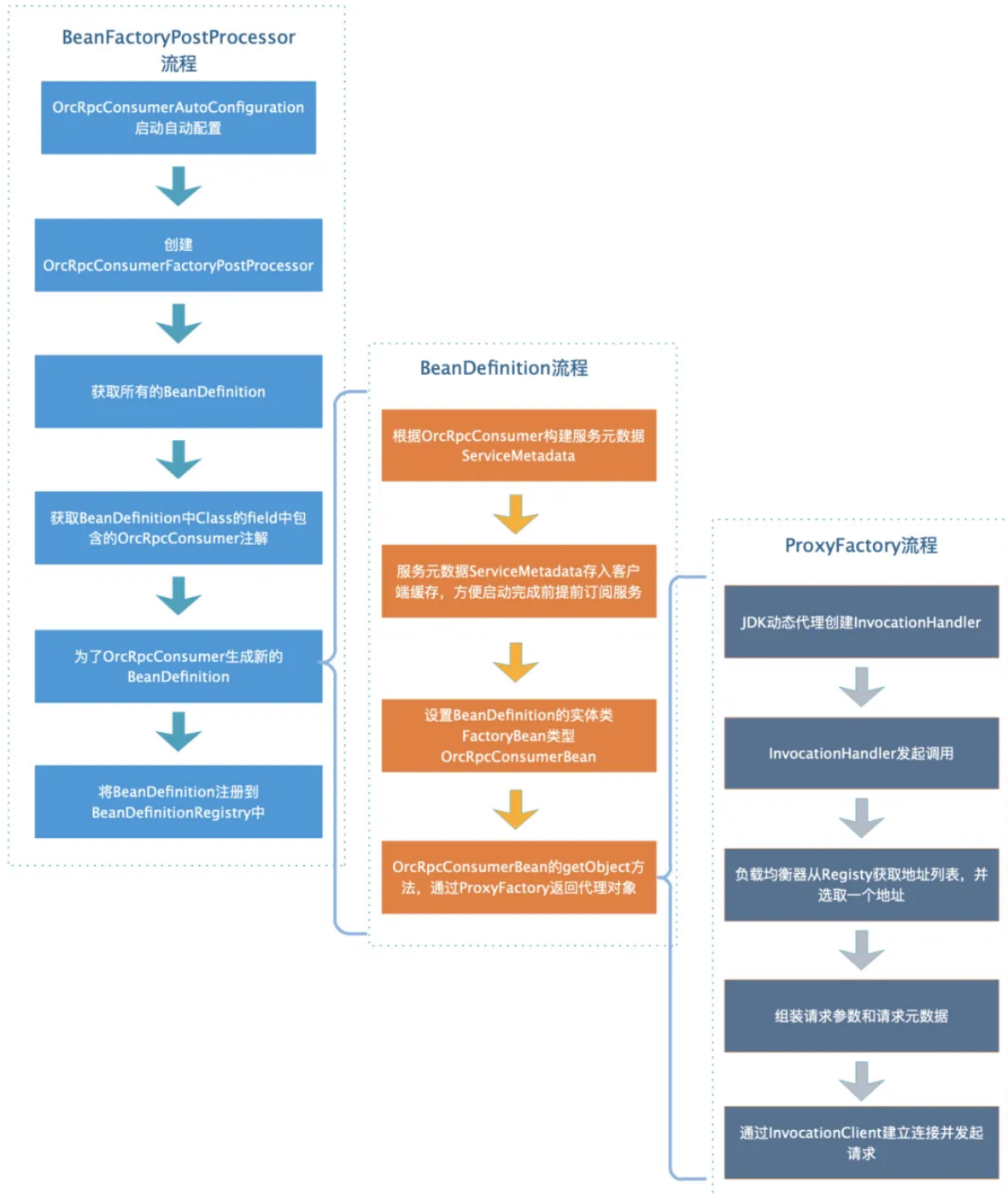
上面监听Spring容器初始化事件时注意，由于Spring包含多个容器，如web容器和核心容器，他们还有父子关系，为了避免重复执行注册，只处理顶层的容器即可。

3 服务消费方 Service Consumer

服务消费方需要在应用启动完成前为依赖的服务创建好代理对象，这里有很多种方法，常见的有两种：

- 一是在应用的Spring Context初始化完成事件时触发，扫描所有的Bean，将Bean中带有OrcRpcConsumer注解的field获取到，然后创建field类型的代理对象，创建完成后，将代理对象set给此field。后续就通过该代理对象创建服务端连接，并发起调用。
- 二是通过Spring的BeanFactoryPostProcessor，其可以对bean的定义BeanDefinition(配置元数据)进行处理；Spring IOC会在容器实例化任何其他bean之前运行BeanFactoryPostProcessor读取BeanDefinition，可以修改这些BeanDefinition，也可以新增一些BeanDefinition。

本实现也采用第二种方式，处理流程如下：



BeanFactoryPostProcessor的主要实现：

```

@Override
public void postProcessBeanFactory(ConfigurableListableBeanFactory
beanFactory)
throws BeansException {
this.beanFactory = beanFactory;
  
```

```

        postProcessRpcConsumerBeanFactory(beanFactory,
            (BeanDefinitionRegistry)beanFactory);
    }

    private void
    postProcessRpcConsumerBeanFactory(ConfigurableListableBeanFactory beanFactory,
        BeanDefinitionRegistry beanDefinitionRegistry) {
        String[] beanDefinitionNames = beanFactory.getBeanDefinitionNames();
        int len = beanDefinitionNames.length;
        for (int i = 0; i < len; i++) {
            String beanDefinitionName = beanDefinitionNames[i];
            BeanDefinition beanDefinition =
            beanFactory.getBeanDefinition(beanDefinitionName);
            String beanClassName = beanDefinition.getBeanClassName();
            if (beanClassName != null) {
                Class<?> clazz = ClassUtils.resolveClassName(beanClassName,
                    classLoader);
                ReflectionUtils.doWithFields(clazz, new FieldCallback() {
                    @Override
                    public void doWith(Field field) throws
                    IllegalArgumentException, IllegalAccessException {
                        parseField(field);
                    }
                });
            }
        }

        Iterator<Entry<String, BeanDefinition>> it =
        beanDefinitions.entrySet().iterator();
        while (it.hasNext()) {
            Entry<String, BeanDefinition> entry = it.next();
            if (context.containsBean(entry.getKey())) {
                throw new IllegalArgumentException("Spring context already has a
                bean named " + entry.getKey());
            }
            beanDefinitionRegistry.registerBeanDefinition(entry.getKey(),
            entry.getValue());
            log.info("register OrcRpcConsumerBean definition: {}",
            entry.getKey());
        }

        private void parseField(Field field) {
            // 获取所有OrcRpcConsumer注解
            OrcRpcConsumer orcRpcConsumer =
            field.getAnnotation(OrcRpcConsumer.class);
            if (orcRpcConsumer != null) {
                // 使用field的类型和OrcRpcConsumer注解一起生成BeanDefinition
                OrcRpcConsumerBeanDefinitionBuilder beanDefinitionBuilder = new
                OrcRpcConsumerBeanDefinitionBuilder(field.getType(), orcRpcConsumer);
                BeanDefinition beanDefinition = beanDefinitionBuilder.build();
                beanDefinitions.put(field.getName(), beanDefinition);
            }
        }
    }

```

ProxyFactory的主要实现：

```
public class JdkProxyFactory implements ProxyFactory{

    @Override
    public Object getProxy(ServiceMetadata serviceMetadata) {
        return Proxy
            .newProxyInstance(serviceMetadata.getClass().getClassLoader(), new
Class[] {serviceMetadata.getClass()},
            new ClientInvocationHandler(serviceMetadata));
    }

    private class ClientInvocationHandler implements InvocationHandler {

        private ServiceMetadata serviceMetadata;

        public ClientInvocationHandler(ServiceMetadata serviceMetadata) {
            this.serviceMetadata = serviceMetadata;
        }

        @Override
        public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
            String serviceId = ServiceUtils.getServiceId(serviceMetadata);
            // 通过负载均衡器选取一个服务提供方地址
            ServiceURL service =
InvocationServiceSelector.select(serviceMetadata);

            OrcRpcRequest request = new OrcRpcRequest();
            request.setMethod(method.getName());
            request.setParameterTypes(method.getParameterTypes());
            request.setParameters(args);
            request.setRequestId(UUID.randomUUID().toString());
            request.setServiceId(serviceId);

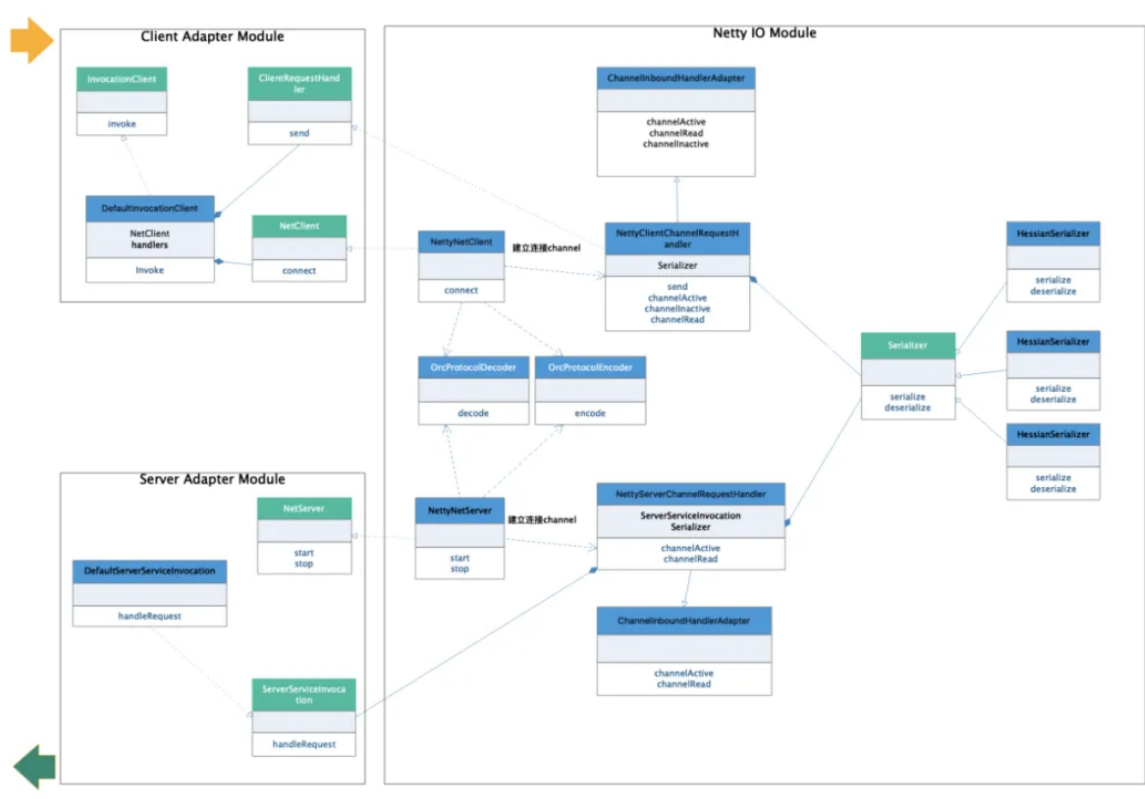
            OrcRpcResponse response =
InvocationClientContainer.getInvocationClient(service.getServerNet()).invoke(requ
est, service);
            if (response.getStatus() == RpcStatusEnum.SUCCESS) {
                return response.getData();
            } else if (response.getException() != null) {
                throw new OrcRpcException(response.getException().getMessage());
            } else {
                throw new OrcRpcException(response.getStatus().name());
            }
        }
    }
}
```

本实现只使用JDK动态代理，也可以使用cglib或Javassist实现以获得更好的性能，JdkProxyFactory中。

4 IO模块



UML图如下：



结构比较清晰，分三大模块：客户端调用适配模块、服务端请求响应适配模块和Netty IO服务模块。

客户端调用适配模块

此模块比较简单，主要是为客户端调用时建立服务端接，并将连接存入缓存，避免后续同服务调用重复建立连接，连接建立成功后发起调用。下面是DefaultInvocationClient的实现：

```

public class DefaultInvocationClient implements InvocationClient {

    private static DefaultInvocationClient INSTANCE_TCP;

    static {
        INSTANCE_TCP = new DefaultInvocationClient(new NettyNetClient());
    }

    private NetClient orcRpcClient;

    private Map<String, Serializer> supportSerializerMap;

    private DefaultInvocationClient() {

    }

    private DefaultInvocationClient(NetClient orcRpcClient) {
        this.orcRpcClient = orcRpcClient;
        init();
    }

    private void init() { supportSerializerMap = SpiLoaderUtils.getSupportSerializer(); }

    // todo only netty tcp
    public static DefaultInvocationClient getInstance(String serverNet) { return INSTANCE_TCP; }

    @Override
    public OrcRpcResponse invoke(OrcRpcRequest orcRpcRequest, ServiceURL serviceURL) {
        String address = serviceURL.getAddress();

        ClientRequestHandler handler = ClientServiceAddressHandlerCache.get(address);
        if (handler == null) {
            Serializer serializer = supportSerializerMap.get(serviceURL.getSerializer());
            handler = orcRpcClient.connect(address, serializer);
            log.debug("establish new channel");
        }
        return handler.send(orcRpcRequest);
    }
}

```

从缓存中获取某服务端连接的channel handler

缓存中没有重新建立连接

使用handler发送请求

服务端请求响应适配模块

服务请求响应模块也比较简单，是根据请求中的服务名，从缓存中获取服务元数据，然后从请求中获取调用的方法和参数类型信息，反射获取调用方法信息。然后从spring context中获取bean进行反射调用。

```

public class DefaultServerServiceInvocation implements ServerServiceInvocation {

    public DefaultServerServiceInvocation() {

    }

    @Override
    public OrcRpcResponse handleRequest(OrcRpcRequest req) throws Exception {
        // 获取服务元数据
        ServiceMetadata serviceMetadata = ServerServiceMetadataCache.get(req.getServiceId());
        OrcRpcResponse response = null;
        if (serviceMetadata == null) {
            response = new OrcRpcResponse(RpcStatusEnum.NOT_FOUND);
        } else {
            try {
                // 反射调用
                Method method = serviceMetadata.getClazz().getMethod(req.getMethod(), req.getParameterTypes());
                Object returnValue = method.invoke(BeanContext.getBean(serviceMetadata.getClazz()), req.getParameters());
                response = new OrcRpcResponse(RpcStatusEnum.SUCCESS);
                response.setData(returnValue);
            } catch (Exception e) {
                response = new OrcRpcResponse(RpcStatusEnum.ERROR);
                response.setException(e);
            }
        }
        // 填充请求id
        response.setRequestId(req.getRequestId());

        return response;
    }
}

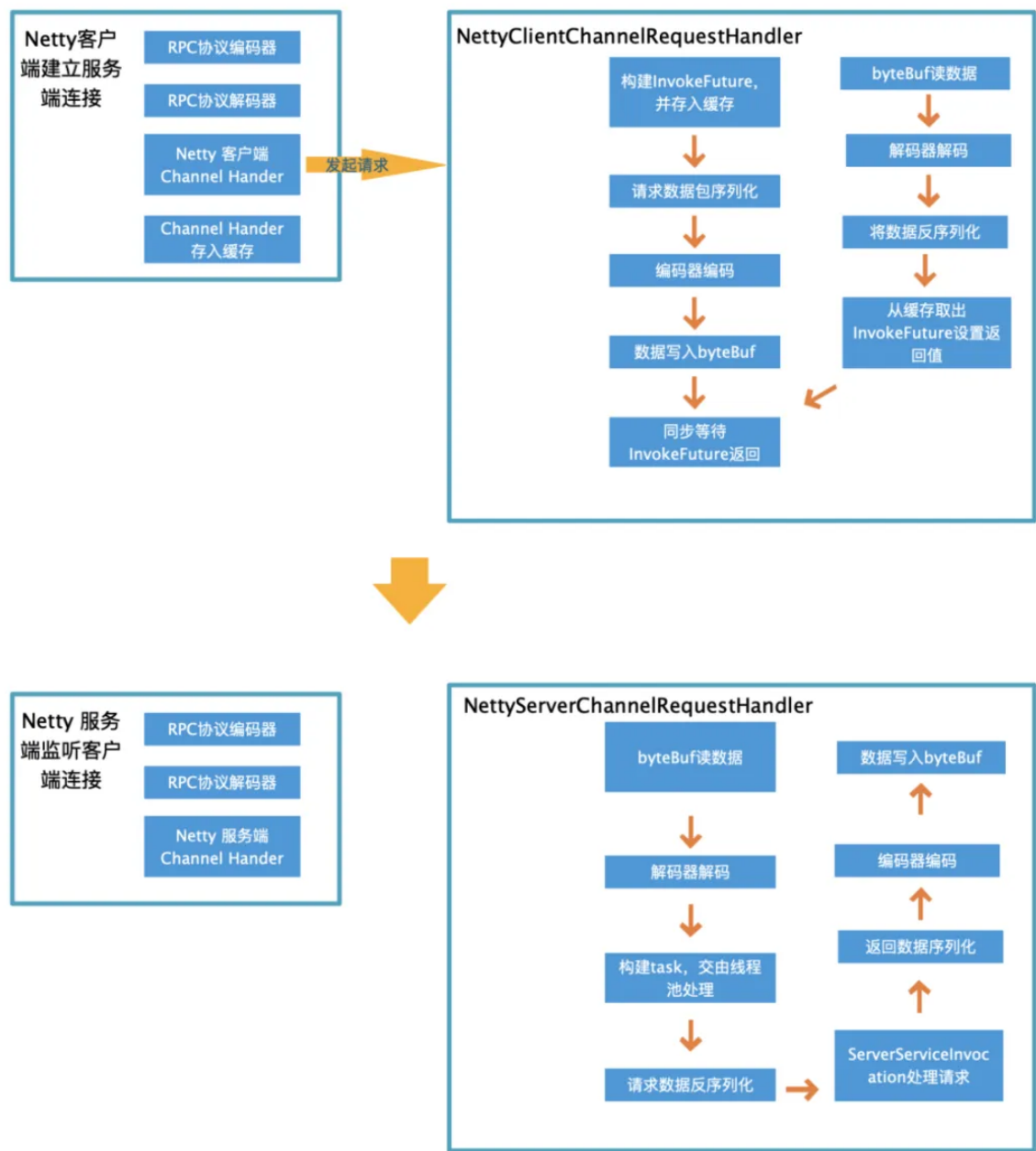
```

从服务端缓存中获取服务元数据

获取bean 反射调用

Netty IO服务模块

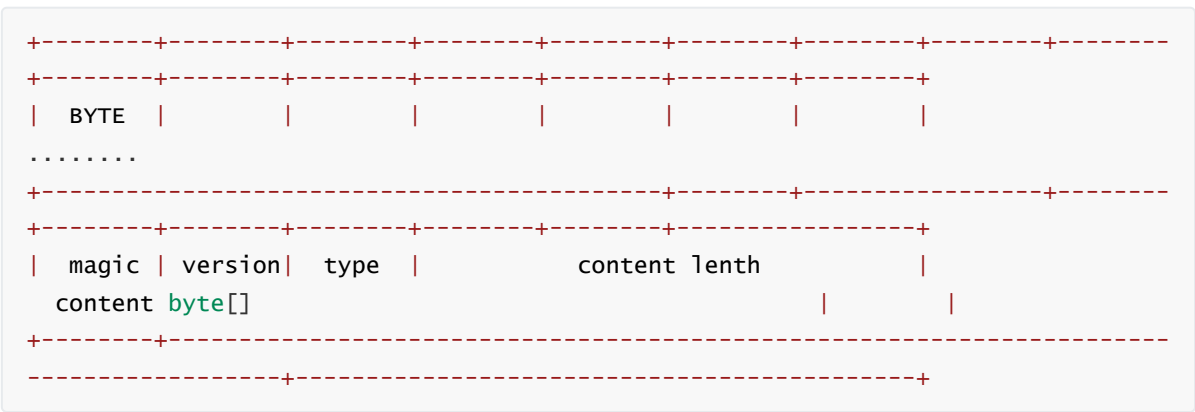
Netty IO服务模块是核心，稍复杂一些，客户端和服务端主要处理流程如下：



其中，重点是这四个类的实现：NettyNetClient、NettyNetServer、NettyClientChannelRequestHandler和NettyServerChannelRequestHandler，上面的UML图和下面流程图基本上讲清楚了它们的关系和一次请求的处理流程，这里就不再展开了。

下面重点讲一下编码解码器。

在技术选型章节中，提及了采用的通信协议，定义了私有的RPC协议：



- 第一个字节是魔法数定义为0X35。
- 第二个字节代表协议版本号。
- 第三个字节是请求类型，0代表请求1代表响应。
- 第四个字节表示消息长度，即此四个字节后面此长度的内容是消息content。

编码器的实现如下：

```
@Override
protected void encode(ChannelHandlerContext channelHandlerContext, ProtocolMsg
protocolMsg, ByteBuf byteBuf)
    throws Exception {
    // 写入协议头
    byteBuf.writeByte(ProtocolConstant.MAGIC);
    // 写入版本
    byteBuf.writeByte(ProtocolConstant.DEFAULT_VERSION);
    // 写入请求类型
    byteBuf.writeByte(protocolMsg.getMsgType());
    // 写入消息长度
    byteBuf.writeInt(protocolMsg.getContent().length);
    // 写入消息内容
    byteBuf.writeBytes(protocolMsg.getContent());
}
```

解码器的实现如下：

```
/**
 * 协议开始的标志 magic + version + type + length 占据7个字节
 */
public final int BASE_LENGTH = 7;

@Override
protected void decode(ChannelHandlerContext channelHandlerContext, ByteBuf
byteBuf, List<Object> list)
    throws Exception {
    // 可读字节小于基本长度，无法解析出payload长度，返回
    if (byteBuf.readableBytes() < BASE_LENGTH) {
        return;
    }
    // 记录包头开始的index
    int beginIndex;
    while (true) {
        // 记录包头开始的index
        beginIndex = byteBuf.readerIndex();
        // 标记包头开始的index
        byteBuf.markReaderIndex();
        // 读到了协议头魔数，结束循环
        if (byteBuf.readByte() == ProtocolConstant.MAGIC) {
            break;
        }
        // 未读到包头，略过一个字节
        // 每次略过一个字节，去读取包头信息的开始标记
        byteBuf.resetReaderIndex();
    }
}
```

```

        byteBuf.readByte();

        /**
         * 当略过，一个字节之后，数据包的长度，又变得不满足
         * 此时结束。等待后面的数据到达
         */
        if (byteBuf.readableBytes() < BASE_LENGTH) {
            return;
        }
    }
    // 读取版本号
    byte version = byteBuf.readByte();
    // 读取消息类型
    byte type = byteBuf.readByte();
    // 读取消息长度
    int length = byteBuf.readInt();
    // 判断本包是否完整
    if (byteBuf.readableBytes() < length) {
        // 还原读指针
        byteBuf.readerIndex(beginIndex);
        return;
    }
    byte[] data = new byte[length];
    byteBuf.readBytes(data);

    ProtocolMsg msg = new ProtocolMsg();
    msg.setMsgType(type);
    msg.setContent(data);
    list.add(msg);
}

```

六 测试

机器配置为4核i5，16g内存，使用Nacos注册中心，启动一个服务器，一个客户端情况下，采用轮询负载均衡策略的情况下，使用Apache ab测试。

在启用8个线程发起10000个请求的情况下，可以做到18秒完成所有请求，qps550：

```
192:~ orc$ ab -c 8 -n 10000 http://localhost:9001/product/getById?id=1
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

Benchmarking localhost (be patient)

```
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests
```

8 个线程发起 10000 个请求

```
Server Software:
Server Hostname: localhost
Server Port: 9001

Document Path: /product/getById?id=1
Document Length: 48 bytes

Concurrency Level: 8
Time taken for tests: 18.073 seconds
Complete requests: 10000
Failed requests: 0
Total transferred: 1530000 bytes
HTML transferred: 480000 bytes
Requests per second: 553.31 [#/sec] (mean)
Time per request: 14.458 [ms] (mean)
Time per request: 1.807 [ms] (mean, across all concurrent requests)
Transfer rate: 82.67 [Kbytes/sec] received
```

Connection Times (ms)

	min	mean[+/-sd]	median	max
Connect:	0	4 9.9	3	644
Processing:	1	9 17.2	5	652
Waiting:	1	7 14.3	4	650
Total:	1	13 20.7	9	669

Percentage of the requests served within a certain time (ms)

50%	9
66%	17
75%	21
80%	22
90%	27
95%	30
98%	34
99%	38
100%	669 (longest request)

在启用100个线程发起10000个请求的情况下，可以做到13.8秒完成所有请求，qps724:

```
192:~ orc$ ab -c 100 -n 10000 http://localhost:9001/product/getById?id=1
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests

Server Software:
Server Hostname:      localhost
Server Port:          9001

Document Path:        /product/getById?id=1
Document Length:      48 bytes

Concurrency Level:    100
Time taken for tests:  13.798 seconds
Complete requests:    10000
Failed requests:       0
Total transferred:    1530000 bytes
HTML transferred:     480000 bytes
Requests per second:  724.73 [#/sec] (mean)
Time per request:     137.983 [ms] (mean)
Time per request:     1.380 [ms] (mean, across all concurrent requests)
Transfer rate:        108.28 [Kbytes/sec] received

Connection Times (ms)
              min    mean[+/-sd] median    max
Connect:        0     56  57.9      37     680
Processing:      2     81  71.6      53     730
Waiting:         1     75  64.5      50     730
Total:           7    137 109.1      93     853

Percentage of the requests served within a certain time (ms)
 50%    93
 66%   180
 75%   213
 80%   231
 90%   270
 95%   280
 98%   310
 99%   344
100%   853 (longest request)
```

100个线程发起10000个请求

七 总结

在实现这个RPC框架的过程中，我们可以学习到很多知识，比如通信协议、IO框架等。也横向学习了当前最热的gRPC